

Computer Organization and Architecture (CSL251)
LAB-1
2025-26 W

Installing the Spike RISC-V Simulator for Linux

Install Git

- Git is required to clone source code repositories from platforms like GitHub.
- Run the following command in the terminal:

sudo apt install git

- After installation, you can verify Git by running:

git --version

Clone the Spike (RISC-V ISA Simulator) Repository

- Now, we download the Spike RISC-V ISA Simulator source code from GitHub using Git.
- Run the following commands:

  git clone https://github.com/riscv-software-src/riscv-isa-sim.git spike-demo

- After installation, open directory by:

  cd spike-demo

Install Required Dependencies for Spik

- Spike requires several system libraries to build and run correctly.
- Install them using the following command:

sudo apt-get install device-tree-compiler

Create Build Directory

- We now create a separate build directory and configure the Spike build environment.
- Run the following commands:

mkdir build

cd build

Configure Spike

- Configure the Spike build environment.
- Run the following commands:

../configure --prefix=/home/deepesh/riscv-demo

- Write your username instead of <deepesh>.

Build Spike (Compile the Source Cod

- Now we compile the Spike simulator from source.
- Run the following command:

make -j16

- You can use -j$(nproc) → automatically uses all available CPU cores

Install Spike Simulator

- After successful compilation, install the Spike simulator to the specified directory.
- Run the following command:

sudo make install

- make install - Copies the compiled binaries, libraries, and support files to the directory specified by --prefix

Verify RISC-V Cross Compiler Installati

- Before running RISC-V programs, we need to ensure that the RISC-V cross-compiler is installed correctly.
- Run the following command:

riscv64-unknown-elf-gcc -v

- **riscv64-unknown-elf-gcc** GCC cross-compiler targeting 64-bit RISC-V architecture

Verify Spike Installation and Explore Optio

- After installation, you can check if Spike is working correctly and see all available options:
- Run the following command:

./spike --help

- **./spike -** Runs the Spike executable in the current directory (or use full path if installed elsewhere)

Compile a Sample RISC-V Progra

- Now we will create a directory for your RISC-V programs, compile a simple C program, and check the generated binary.
- Create a directory for your programs:

mkdir ~/riscv-programs

cd ~/riscv-programs

- **Open a text editor and create a file called hello.c:**

nano hello.c

Compile a Sample RISC-V Progra

- Type the following code inside hello.c:

```
#include <stdio.h>
int main() {
    printf("Hello, RISC-V World!\n");
    return 0;
}
```

Save the file and exit nano:

- Press Ctrl + O → press Enter to save
- Press Ctrl + X → to exit the editor

Compile a Sample RISC-V Progra

- Compile the Hello World program for RISC-V:

riscv64-unknown-elf-gcc hello.c -o hello

- Verify the binary:

file hello

Clone and Set Up the RISC-V Proxy Kernel (

- The RISC-V Proxy Kernel (PK) is needed to run programs on Spike that require a minimal operating system interface.
- Move back to your working directory

cd ../..

- Clone the PK repository from GitHub:

git clone https://github.com/riscv-software-src/riscv-pk.git pk-demo

- Enter the PK directory:

cd pk-demo

Create Build Directory

- We now create a separate build directory
- Run the following commands:

mkdir build

cd build

Configure the Proxy Kernel (PK) Buil

- Before building PK, we need to configure it with the appropriate installation directory and target architecture.
- Run the following commands:

../configure --prefix=/home/deepesh/riscv-demo --host=riscv64-unknown-elf

- Write your username instead of <deepesh>.

Build, Install, and Verify the Proxy Kernel (

- Compile PK:

make

- Install PK to the specified directory:

sudo make install

- After installation, the PK binary should be available at:

/home/umesh/riscv-demo/riscv64-unknown-elf/bin/pk

Run a RISC-V Program on Spike and Configure Env

- Run your program on Spike using the Proxy Kernel (PK):

spike ~/riscv-demo/riscv64-unknown-elf/bin/pk a.out

- Add Spike build directory to your PATH permanently:

echo 'export PATH=$PATH:$HOME/Desktop/spike-demo/build'
>> ~/.zshrc

source ~/.zshrc

Run a RISC-V Program on Spike and Configure Env

- Copy PK to the current working directory

cp /home/umesh/riscv-demo/riscv64-unknown-elf/bin/pk .

- Run the program using the local PK:

spike pk a.out